YARA Rules Comprehensive Documentation

Complete Category, Sub-Category, and Rule Analysis

This document provides comprehensive documentation of all YARA rules in the documented folder, including main categories, sub-categories, their working mechanisms, and detailed information about each rule type.

Main Categories (14 Total)

#	Category	Description	Sub-Categories
1	sql_injection	SQL injection attacks	SQL_Brute_Force
2	scripting_attacks	Malicious script-based attacks	PowerShell_Abuse, Scripting_Attacks
3	brute_force	Credential brute forcing	SQL_Brute_Force, BruteRAT
4	credential_theft	Credential stealing malware	Infostealers, Keyloggers, Banking Trojans
5	phishing	Phishing campaigns	Phishing_Lures
6	behavioral	Behavioral detection rules	Evasion_Sandbox
7	rootkit	Rootkit and kernel-level threats	Rootkits_Kernel
8	malware	General malware	All sub-categories
9	trojans	Trojan horses	All sub-categories
10	ransomware	Ransomware families	Crypto_Ransomware
11	spyware	Spyware and surveillance	Keyloggers, Infostealers
12	worms	Self-replicating malware	Rootkits_Kernel (contains worms)
13	autorun	Auto-run based malware	Persistence_Methods
14	security	Security monitoring rules	Various

Sub-Categories Overview

Total Rule Count by Sub-Category

Sub-Category	Rule Count	Primary Threat Type
Infostealers	10,807	Credential Theft
Downloaders_Droppers	9,447	Payload Delivery
RATs	4,345	Remote Access
Banking_Trojans	4,039	Financial Fraud
Crypto_Ransomware	2,367	File Encryption
Keyloggers	1,236	Surveillance
PowerShell_Abuse	1,180	Script-Based Attacks
Persistence_Methods	1,169	Persistence
Rootkits_Kernel	941	Rootkit/Kernel
SQL_Brute_Force	3	Database Attacks
Downloaders_Droppers	9,447	Downloaders
Injection_Techniques	219	Code Injection
Evasion_Sandbox	145	Sandbox Evasion
AMSI_Bypass	37	Security Bypass
Phishing_Lures	212	Phishing

Detailed Sub-Category Analysis

---

1. AMSI Bypass (`AMSI_Bypass`)

Rule Count: 37

Purpose: Detects techniques used to bypass Windows Defender AMSI (Antimalware Scan Interface)

Working Mechanism:

- Targets scripts designed to patch `AmsiScanBuffer` in memory
- Detects assembly code patterns that disable scanning providers
- Identifies specific function calls used to bypass security

Techniques Detected:

- `AmsiScanBuffer` patching
- `Assembly.GetType()` manipulation
- `.getfield` method abuse
- Memory patching for security evasion

Example Rule: AmsiBypazz

```
rule Trojan_Win32_AmsiBypazz_A_MTB{
    strings :
        $a_00_0 = {5b 00 52 00 65 00 66 00 5d 00 2e 00 41 00 73 00 73 00
65 00 6d 00 62 00 6c 00 79 00 2e 00 47 00 65 00 74 00 54 00 79 00 70 00 65
00 } //[Ref].Assembly.GetType
        $a_00_1 = {2e 00 67 00 65 00 74 00 66 00 69 00 65 00 6c 00 64 00 }
//.getfield
    condition:
        ((#a_00_0 & 1)*1+(#a_00_1 & 1)*1) >=2
}
```

Malware Families Detected:

- AmsiBypazz
- AmsiPatch
- AmsiBypass
- AmsiTamper
- MalAmsiExec
- ShellcodeRunner
- Redcap
- PsAttack

Implementation:

- Monitor AMSI event logs
- Deploy on EDR for memory scanning
- Alert on PowerShell/WScript execution with unusual behavior

---

2. Banking Trojans (`Banking_Trojans`)

Rule Count: 4,039

Purpose: Detects financial fraud malware targeting banking credentials and transactions

Working Mechanism:

- Targets "Web Injections" - fake forms injected into legitimate banking sites
- Captures transaction authentication numbers (TAN)
- Intercepts 2FA/mFA codes
- Keylogging for banking credentials

Techniques Detected:

- Browser hooking
- Web injection templates
- Man-in-the-browser (MITB)
- Screen scraping
- Form grabbing

Example Rule: Android Malware Banker

```
rule Android_Malware : iBanking android
{
    strings:
        $pk = {50 4B}
        $file1 = "AndroidManifest.xml"
        $file2 = "res/drawable-xxhdpi/ok_btn.jpg"
        $string1 = "bot_id"
        $string2 = "type_password2"
    condition:
        ($pk at 0 and 2 of ($file*) and ($string1 or $string2))
}
```

Malware Families Detected:

- iBanking (Android)
- Pony2000
- FakeMosKow
- Various Android banker variants

Implementation:

- Deploy on endpoint security solutions
- Monitor network traffic to known C2 servers
- Scan for suspicious browser extensions

---

3. Crypto Ransomware (`Crypto_Ransomware`)

Rule Count: 2,367

Purpose: Detects file-encryption ransomware threats

Working Mechanism:

- Targets the encryption loop (iterating through files)
- Detects AES/RSA cryptographic operations
- Identifies ransom note creation patterns
- Monitors for file extension modifications

Techniques Detected:

- File encryption routines
- Key generation patterns
- Ransom note dropped (README, LOCKED, etc.)
- Volume shadow deletion
- MFT manipulation

Malware Families Detected:

- Conti
- LockFile
- Ramsil
- Various crypto ransomware variants

Implementation:

- Monitor for mass file modifications
- Alert on .locked, .encrypted, .crypt extensions
- Monitor VSSadmin usage

---

4. Downloaders & Droppers (`Downloaders_Droppers`)

Rule Count: 9,447

Purpose: Detects lightweight malware designed to download and execute additional payloads

Working Mechanism:

- Identifies binaries that connect to C2 servers
- Detects HTTP/HTTPS requests to known malicious domains
- Flags payloads that download and execute secondary malware
- Analyzes URL patterns and download mechanisms

Techniques Detected:

- C2 communication
- URL downloading
- Payload execution
- DLL injection
- Process hollowing

Implementation:

- Scan browser cache directories
- Monitor temp folders
- Watch AppData directories for new executables

---

5. Evasion & Sandbox Detection (`Evasion_Sandbox`)

Rule Count: 145

Purpose: Detects malware that attempts to evade analysis by sandboxes and VMs

Working Mechanism:

- Identifies code checking for VM-specific artifacts
- Detects debugger detection techniques
- Flags checks for low file count (sandbox detection)
- Identifies timing attacks to detect analysis

Techniques Detected:

- VM detection (VBox, VMware, QEMU)
- Sandbox detection
- Debugger detection (IsDebuggerPresent)
- Timing checks (sleep evasion)
- CPU instruction checks

Example Artifacts Detected:

- `VBoxGuest.sys`
- `vmtoolsd.dll`
- `VMWareTray.exe`
- VirtualBox registry keys
- Low user count detection

Implementation:

- Deploy in malware analysis sandboxes
- Flag samples attempting to evade analysis
- Use generic detection for evasive samples

---

6. Infostealers (`Infostealers`)

Rule Count: 10,807

Purpose: Detects industrial-scale data theft malware

Working Mechanism:

- Targets browser data theft (saved passwords, cookies)
- Searches for files containing "password"
- Harvests session cookies for MFA bypass
- Exfiltrates sensitive documents

Techniques Detected:

- Browser profile scanning
- Cookie harvesting
- Credential database extraction
- File system searching
- Clipboard monitoring

Example Rule: ACRStealer

```
rule Trojan_Win32_ACRStealer_DA_MTB{
    strings :
        $a_80_0 = {75 73 65 72 5f 70 72 65 66} //user_pref
        $a_80_1 = {3c 64 69 73 63 61 72 64 65 64 3e} //<discarded>
        $a_80_2 = {73 74 65 61 6d 63 6f 6d 6d 75 6e 69 74 79 2e 63 6f 6d}
//steamcommunity.com
        $a_80_3 = {43 72 65 61 74 65 54 6f 6f 6c 68 65 6c 70 33 32 53 6e
61 70 73 68 6f 74} //CreateToolhelp32Snapshot
    condition:
        ((#a_80_0 & 1)*1+(#a_80_1 & 1)*1+(#a_80_2 & 1)*1+(#a_80_3 & 1)*1)
>=4
}
```

Data Targets:

- Chrome/Firefox saved passwords
- Browser cookies

- Steam credentials
- Cryptocurrency wallets
- Document files containing "password"

Implementation:

- Deploy on workstations
- Monitor for suspicious browser process access
- Alert on unusual file access patterns

---

7. Injection Techniques (`Injection_Techniques`)

Rule Count: 219

Purpose: Detects stealthy code injection and process manipulation techniques

Working Mechanism:

- Identifies Process Hollowing (replacing legitimate process memory)
- Detects Reflective DLL Loading (loading from memory, not disk)
- Flags Remote Thread Injection
- Monitors for APC injection

Techniques Detected:

- Process Hollowing
- Reflective DLL Loading
- Remote Thread Injection
- QueueUserAPC injection
- Process Doppelgänging

Example Rule: Bruterat

```
rule VirTool_Win32_Bruterat_B{
    strings :
        $a_80_5 = {5d 20 45 6c 65 76 61 74 65 64 } //] Elevated
        $a_80_6 = {5d 20 49 6e 6a 65 63 74 65 64 } //] Injected
        $a_80_7 = {5d 20 53 70 6f 6f 66 65 64 } //] Spoofed
        $a_80_12 = {5d 20 49 6d 70 65 72 73 6f 6e 61 74 65 64 } //]
Impersonated
    condition:
        ((#a_80_5 & 1)*1+(#a_80_6 & 1)*1+(#a_80_7 & 1)*1) >=3
}
```

Implementation:

- Integrate with EDR memory scanning
- Monitor for code in unexpected memory regions
- Alert on suspicious process creation

---

8. Keyloggers (`Keyloggers`)

Rule Count: 1,236

Purpose: Detects real-time keystroke recording surveillance malware

Working Mechanism:

- Detects installation of global keyboard hooks
- Identifies `SetWindowsHookEx` API calls
- Flags creation of hidden log files
- Monitors for keystroke file creation

Techniques Detected:

- Global keyboard hooks (SetWindowsHookEx)
- Low-level keyboard hooks (WH_KEYBOARD_LL)
- Journal hook installation
- Hidden file creation for logs

Implementation:

- Monitor for SetWindowsHookEx calls
- Alert on new files in hidden locations
- Watch for unusual process access to keyboard drivers

---

9. Persistence Methods (`Persistence_Methods`)

Rule Count: 1,169

Purpose: Detects how malware achieves persistence across reboots

Working Mechanism:

- Monitors Registry Run keys
- Detects Scheduled Task creation
- Identifies WMI event consumer abuse

- Flags startup folder modifications

Techniques Detected:

- Registry Run keys (HKCU/HKLM Run)
- Scheduled Tasks creation
- WMI persistence
- Startup folder persistence
- DLL search order hijacking

Common Registry Keys Monitored:

- `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKLM\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce`

Implementation:

- Periodically scan Windows Registry
- Monitor Task Scheduler definitions
- Alert on new scheduled tasks

---

10. Phishing Lures (`Phishing_Lures`)

Rule Count: 212

Purpose: Detects the "lure" phase of phishing campaigns

Working Mechanism:

- Targets email headers from phishing kits
- Detects social engineering keywords
- Identifies malicious document templates
- Flags fake login pages

Techniques Detected:

- Email header manipulation
- Social engineering keywords
- Malicious Office documents
- Fake credential prompts

Keywords Detected:

- "urgent payment"

- "O365 security alert"
- "account suspended"
- "verify your identity"

Implementation:

- Deploy on Email Gateways (SEG)
- Scan temporary download folders
- Monitor for suspicious document execution

---

11. PowerShell Abuse (`PowerShell_Abuse`)

Rule Count: 1,180

Purpose: Detects malicious use of PowerShell for attacker operations

Working Mechanism:

- Identifies Base64 encoded commands
- Detects obfuscation patterns
- Flags common attacker toolkits
- Monitors for encoded script execution

Techniques Detected:

- Base64 encoded commands
- Character replacement obfuscation
- Backtick obfuscation
- Empire/PowerSploit scripts

Example Obfuscation Patterns:

- `$env:Var` environment variable abuse
- `-enc` encoded command flag
- `IEX` (Invoke-Expression) with encoded scripts
- Download cradle patterns

Toolkits Detected:

- PowerShell Empire
- PowerSploit
- Nishang
- Covenant

Implementation:

- Monitor via AMSI logs
- Enable command-line auditing
- Deploy PowerShell transcription

---

12. RATs (Remote Access Trojans) (`RATs`)

Rule Count: 4,345

Purpose: Detects interactive attacker control malware

Working Mechanism:

- Targets command structures of RAT families
- Identifies internal strings and C2 protocols
- Detects keylogging and remote control features
- Flags reverse shell functionality

Malware Families Detected:

- NjRAT
- Remcos
- DarkComet
- QuasarRAT
- AsyncRAT
- XenoRAT

Capabilities Detected:

- Remote desktop control
- File transfer
- Keylogging
- Command execution
- Persistence mechanisms
- C2 communication

Implementation:

- Critical for detecting active intruders
- Monitor for unusual network connections
- Alert on suspicious process behavior

---

13. Rootkits & Kernel Implants (`Rootkits_Kernel`)

Rule Count: 941

Purpose: Detects deep-seated invisibility and kernel-level threats

Working Mechanism:

- Targets malicious drivers
- Detects boot-level code hiding
- Identifies kernel mode rootkits
- Flags system hook manipulation

Techniques Detected:

- Kernel mode drivers
- Boot kit functionality
- Inline hooks
- SSDT hooks
- DKOM (Direct Kernel Object Manipulation)

Malware Types Detected:

- Rootkits
- Worms (self-replicating)
- Kernel implants
- Boot kits

Example Worms Detected:

- Yimfoca
- Voterai
- VB
- Ultarmine
- Slimbraju
- Rootcip
- Rombrast
- Pykspa
- Autorun variants

Implementation:

- Use for low-level forensic scans
- Deploy kernel-level monitoring
- Scan suspect machines with offline tools

---

14. SQL Brute Force (`SQL_Brute_Force`)

Rule Count: 3

Purpose: Detects database-specific credential guessing attacks

Working Mechanism:

- Targets SQL Server authentication attempts
- Identifies brute force patterns
- Flags common SQL passwords
- Detects stored credential patterns

Example Rule: SqlBrute

```
rule Trojan_BAT_SqlBrute_A_MTB{
    strings :
        $a_81_0 = {77 69 6e 6c 6f 67 6f 6e 2e 70 64 62 } //winlogon.pdb
        $a_81_1 = {73 61 40 31 32 33 34 35 36 } //sa@123456
        $a_81_2 = {65 78 65 63 20 73 70 5f 70 61 73 73 77 6f 72 64} //exec
sp_password
    condition:
        ((#a_81_0 & 1)*1+(#a_81_1 & 1)*1+(#a_81_2 & 1)*1) >=3
}
```

Attack Patterns Detected:

- sa@123456 (common SA password)
- SQL stored procedure abuse
- winlogon.Resources SQL files

Implementation:

- Monitor SQL Server logs
- Alert on multiple failed login attempts
- Deploy on database servers

Rule Naming Convention

The YARA rules follow this naming pattern:

```
{category}_{malwareName}_{type}_{platform}_{name}_{variant}.yar
```

Examples:

- `trojans_AmsiBypazz_Trojan_Win32_AmsiBypazz_A_MTB.yar`
- `credential_theft_ACRStealer_Trojan_Win32_ACRStealer_DA_MTB.yar`
- `ransomware_Conti_Ransom_Win64_Conti_A.yar`

Implementation Guide

Where to Deploy Each Sub-Category

Sub-Category	Deployment Location
Phishing_Lures	Email Gateways, Download Folders
PowerShell_Abuse	AMSI Logs, Command-line Auditing
AMSI_Bypass	EDR Memory Scanning
Downloaders_Droppers	Browser Cache, Temp Folders, AppData
Injection_Techniques	EDR Memory Scanners
RATs	Endpoint Detection
Infostealers	Workstation Monitoring
Banking_Trojans	Endpoint + Network Monitoring
Crypto_Ransomware	File System Monitoring
Rootkits_Kernel	Forensic Analysis
Persistence_Methods	Registry + Task Scheduler
Evasion_Sandbox	Malware Analysis Sandboxes
Keyloggers	Endpoint Monitoring
SQL_Brute_Force	Database Server Monitoring

Rule Condition Patterns

Common Condition Types

1. **String Counting:** `(#string_name & 1) >= N`
2. **File Offset:** `$string at 0`
3. **Any/All of:** `any of ($string*)` / `all of ($string*)`

4. **Import Detection:** `import "pe"` then `$some_import in imports`

Summary

This YARA ruleset provides comprehensive coverage of:

- **Initial Access:** Phishing, PowerShell abuse, AMSI bypass
- **Payload Delivery:** Downloaders, droppers
- **Execution:** Injection techniques, process hollowing
- **Persistence:** Registry, scheduled tasks, WMI
- **Stealth:** Rootkits, evasion, sandbox detection
- **Data Theft:** Infostealers, keyloggers, banking trojans
- **Financial:** Ransomware, banking malware
- **Remote Access:** RATs, backdoors
- **Specialized:** SQL brute force

Total Rules Documented: **37,000+**